rag_eval_tasklist.md

```
# RAG 智能问答系统量化评测任务清单（交给 Codex 执行）

## 0. 任务目标

请为当前“基于 Agent 与多级混合检索的 RAG 智能问答系统”实现一套**完整、可复现、可导出简历量化结果**的离线评测流水线，覆盖以下能力：

1. 检索策略收益评测
2. 长文档切分与 Auto-merging 收益评测
3. 查询重写（Step-Back / HyDE / 动态触发）收益评测
4. SSE 流式可观测链路与交互体验评测
5. 端到端 RAG 回答质量与 groundedness 评测

最终目标不是只得到论文式实验结果，而是产出**可以直接写进简历的量化数字**，例如：

- Recall@5 提升 X%
- MRR@10 提升 X%
- 长文档跨段问题准确率提升 X%
- 上下文完整率提升 X%
- 首 token 时间降低 X%
- tracing 覆盖率达到 X%

---

## 1. 总体评测方案

采用如下组合方案：

- **LoTTE**：作为检索主评测集
  用于验证 hybrid retrieval、RRF、rerank 的效果
- **RAGBench**：作为端到端 RAG 补充评测集
  用于验证 answer correctness、groundedness、retrieval relevance
- **自建小规模评测集**：作为项目特性验证集
  用于验证 L1/L2/L3 多级切分、Auto-merging、动态查询重写、SSE 可观测链路

### 1.1 数据集使用目标

#### LoTTE
用于评测：
- dense only vs sparse only vs hybrid retrieval
- RRF 融合收益
- rerank 收益
- 检索延迟

#### RAGBench
用于评测：
- answer correctness
- groundedness
- retrieval relevance
- 端到端 RAG 质量
```

```
#### 自建集
用于评测：
- fixed chunk vs L1/L2/L3 + auto-merging
- Step-Back / HyDE / dynamic rewrite
- cross-chunk 问题表现
- no-answer 场景拒答能力
- SSE trace / TTFT / TTFM

---

## 2. 目录结构要求

请按如下目录组织评测工程：

```text
eval/
  datasets/
    lotte/
    ragbench/
    custom/
  configs/
    retrieval_baselines.yaml
    chunking_baselines.yaml
    generation_baselines.yaml
    latency_eval.yaml
  scripts/
    download_lotte.py
    download_ragbench.py
    build_custom_eval.py
    ingest_corpus.py
    run_retrieval_eval.py
    run_chunking_eval.py
    run_rewrite_eval.py
    run_rag_eval.py
    run_latency_eval.py
    aggregate_results.py
    export_resume_bullets.py
  outputs/
    retrieval/
    chunking/
    rewrite/
    rag/
    latency/
    reports/
  README.md
```

---

3. 最终必须产出的文件

请确保最终生成以下文件：

```
outputs/reports/results_summary.md
outputs/reports/results_table.csv
outputs/reports/retrieval_metrics.csv
outputs/reports/chunking_metrics.csv
```
```

```
outputs/reports/rewrite_metrics.csv
outputs/reports/latency_metrics.csv
outputs/reports/resume_bullets.md
```

其中：

- `results_summary.md`：面向人阅读的总结报告
- `results_table.csv`：总汇总表
- `resume_bullets.md`：自动生成的简历成果表述候选版本

---

4. 数据准备任务

4.1 LoTTE 下载与处理

请完成：

1. 下载 LoTTE benchmark
2. 至少选择 3 个 domain，优先：
   - technology
   - science
   - writing 或 lifestyle
3. 使用 dev / test 集，支持配置切换
4. 统一转换为如下字段结构：

```
{
  "query_id": "xxx",
  "query_text": "xxx",
  "relevant_doc_ids": ["d1", "d2"],
  "domain": "technology"
}
```

1. 将文档语料统一转为当前系统 ingestion 所需格式，支持：
   - fixed chunk
   - L1/L2/L3 多级 chunk

4.2 RAGBench 下载与处理

请完成：

1. 下载 RAGBench 数据集
2. 抽样一个可控子集，默认 300–1000 条
3. 保留字段：

```
{
  "id": "xxx",
  "question": "xxx",
  "context_docs": [...],
  "gold_answer": "xxx",
  "task_type": "xxx"
}
```

1. 转换为当前系统统一问答调用格式

2. 若原始标签中有 explainable labels，请尽量保留到处理后的数据中

4.3 自建评测集构建

请实现 `build_custom_eval.py`，输出 `custom_eval.jsonl`。

样本 schema：

```
{
  "id": "custom_001",
  "question": "...",
  "gold_answer": "...",
  "gold_doc_ids": ["doc_12"],
  "gold_spans": ["..."],
  "question_type": "cross_chunk | direct_fact | ambiguous | no_answer | rewrite_need
ed",
  "needs_parent_context": true,
  "needs_rewrite": true,
  "is_unanswerable": false
}
```

规模要求：

- 总量：40–80 条
- 建议分布：
  - direct_fact: 15
  - cross_chunk: 15
  - rewrite_needed: 10
  - ambiguous: 10
  - no_answer: 10

如果无法自动构造全部数据，请保留一个可手工补充的模板文件，并在 README 中说明补充方式。

---

5. 实验矩阵

5.1 检索策略实验

目标

验证 hybrid retrieval、RRF、rerank 的收益。

对比组

- A1: dense only
- A2: sparse only（BM25）
- A3: dense + sparse + RRF
- A4: dense + sparse + RRF + rerank

数据集

- LoTTE
- 自建集

指标

- Recall@5
- Recall@10
- MRR@10
- NDCG@10
- 平均检索耗时
- P95 检索耗时

输出

- 每条 query 的样本级结果
- 各 variant 的汇总 csv
- 总结 markdown

5.2 长文档切分与 Auto-merging 实验

目标

验证 L1/L2/L3 多级切分与父块动态回取相对于普通 fixed chunk 的收益。

对比组

- B1: fixed chunk，无父块恢复
- B2: L1/L2/L3，多级切分，无 auto-merging
- B3: L1/L2/L3 + auto-merging

数据集

- 自建集
- RAGBench 中长文档或多证据样本子集

指标

- Recall@5
- cross-chunk 问题准确率
- context completeness
- answer groundedness
- 向量库条目数量
- 向量存储占用
- 平均召回 token 数

自定义指标定义

context completeness

若 gold answer 依赖多个相邻证据片段，则返回上下文是否覆盖全部关键证据片段。

5.3 查询重写实验

目标

验证 Step-Back / HyDE / 动态触发 rewrite 的收益。

对比组

- C1: no rewrite
- C2: always Step-Back
- C3: always HyDE
- C4: dynamic rewrite（基于结构化相关性打分门控）

数据集

- 自建集中的 rewrite_needed 子集
- LoTTE 中难样本子集

指标

- Recall@5
- answer accuracy
- groundedness
- 平均额外耗时
- rewrite trigger rate
- rewrite precision

自定义指标定义

rewrite precision

被判定触发 rewrite 的样本中，最终 Recall@5 或 answer accuracy 有提升的比例。

5.4 流式可观测性实验

目标

量化 SSE 流式输出和推理步骤可见化带来的体验收益。

对比组

- D1: 普通同步返回
- D2: SSE 流式，仅返回 final answer
- D3: SSE + 隐式检索/重写步骤推送

数据集

- 自建集中的 20–30 条固定样本
- 每条样本重复跑 3–5 次求均值

指标

- TTFT（首 token 时间）
- TTFM（首条中间事件时间）
- end-to-end latency
- P95 latency
- trace coverage rate
- stage timing breakdown

指标定义

TTFT

从请求发出到第一个 answer token 到达前端的时间

TTFM

从请求发出到第一条可见中间步骤事件到达前端的时间

trace coverage rate

一次请求中，可被成功采集并展示的阶段占比

stage timing breakdown

至少拆分：

- rewrite_ms
- retrieve_ms
- rerank_ms
- generate_ms

5.5 端到端 RAG 评测

目标

从最终回答角度验证系统的 answer correctness 与 groundedness。

对比组

建议至少包含：

- E1: baseline retrieval + baseline generation
- E2: hybrid retrieval + rerank
- E3: hybrid retrieval + rerank + auto-merging
- E4: hybrid retrieval + rerank + auto-merging + dynamic rewrite

数据集

- RAGBench 子集
- 自建集

指标

- answer accuracy
- groundedness
- citation coverage
- refusal correctness（仅 no-answer 样本）
- 平均生成耗时

---

6. 结果记录格式要求

请统一输出样本级 jsonl。

6.1 检索层记录格式

```
{
  "query_id": "xxx",
```

```
  "variant": "hybrid_rrf_rerank",
  "topk_doc_ids": ["d1", "d2", "d3"],
  "recall_at_5": 1,
  "recall_at_10": 1,
  "mrr_at_10": 0.5,
  "ndcg_at_10": 0.71,
  "retrieval_latency_ms": 132
}
```

6.2 生成层记录格式

```
{
  "question_id": "xxx",
  "variant": "auto_merge_dynamic_rewrite",
  "pred_answer": "...",
  "answer_accuracy": 1,
  "groundedness_score": 0.86,
  "citation_coverage": 0.75,
  "is_refusal_correct": null,
  "generation_latency_ms": 840
}
```

6.3 可观测性记录格式

```
{
  "request_id": "xxx",
  "variant": "sse_trace",
  "ttft_ms": 620,
  "ttfm_ms": 210,
  "end_to_end_ms": 2480,
  "trace_coverage_rate": 1.0,
  "stages": {
    "rewrite_ms": 120,
    "retrieve_ms": 95,
    "rerank_ms": 180,
    "generate_ms": 1780
  }
}
```

---

7. 指标计算规则

7.1 Retrieval 指标

- Recall@k：gold doc 是否出现在 top-k
- MRR@10：第一个正确结果的倒数排名
- NDCG@10：排序质量

7.2 Answer accuracy

优先顺序：

1. 若答案是短答案、结构化答案：使用 exact match / token F1
2. 若答案是开放式回答：使用固定 judge prompt 的 LLM-as-a-judge

- 温度固定为 0
- 输出二分类或 0~1 分数
- 需要保留 judge rationale 便于排查

7.3 Groundedness

定义为：回答中的关键结论是否能在返回证据中找到支持。

支持：

- 二分类打分
- 或 0~1 分数

但必须在同一实验中保持一致。

7.4 Refusal correctness

仅在 `is_unanswerable = true` 样本上评估：

- 应拒答且拒答：1
- 应拒答但编造回答：0

---

8. 汇总表要求

请自动生成以下表格。

表 1：检索策略总表

Variant	Recall@5	Recall@10	MRR@10	NDCG@10	Avg Latency	P95 Latency
dense_only						
sparse_only						
hybrid_rrf						
hybrid_rrf_rerank						

表 2：长文档与重写总表

Variant	Cross-chunk Acc	Context Completeness	Groundedness	Answer Acc
fixed_chunk				
multi_level				
auto_merge				
dynamic_rewrite				

表 3：体验与可观测性总表

Variant	TTFT	TTFM	E2E Latency	P95	Trace Coverage
sync		-			0
sse_answer		-			partial
sse_trace					1.0

---

9. 执行顺序要求

请按以下顺序执行，优先快速产出简历可用结果。

第一阶段：先拿到最关键 3 类结果

1. 跑 LoTTE 检索实验

2. 得到：
    - dense only vs hybrid_rrf_rerank 的 Recall@5
    - MRR@10
3. 跑 latency 实验
4. 得到：
    - TTFT
    - P95 latency

第二阶段：补项目差异化结果

1. 跑自建集上的 chunking 实验
2. 得到：
    - cross-chunk accuracy
    - context completeness
3. 跑 rewrite 实验
4. 得到：
    - rewrite 带来的 Recall@5 或 answer accuracy 提升

第三阶段：补端到端 RAG 结果

1. 跑 RAGBench 子集
2. 得到：
    - answer accuracy
    - groundedness
3. 汇总成总报告

---

10. 验收标准

请确保：

1. 所有实验可通过配置切换
2. 固定随机种子
3. 结果可复现
4. 所有结果按 variant 分组聚合
5. 所有实验都支持失败重试
6. 保留异常日志
7. README 说明如何一键运行整套实验
8. 输出一版适合写进简历的成果总结

---

11. README 必须包含的内容

请在 `eval/README.md` 中写清：

1. 环境安装方式
2. 所需 API / 模型配置项
3. 数据下载方式
4. 如何运行：

- retrieval eval
- chunking eval
- rewrite eval
- rag eval
- latency eval

5. 各输出文件位置
6. 如何重现最终报告
7. 如何导出简历成果表述

---

12. 自动生成简历成果表述

请实现 `export_resume_bullets.py`，根据最终结果自动生成至少 2 个版本的“项目成果”描述。

版本 A：偏简历成果

示例模板：

- 基于 LoTTE、RAGBench 与自建评测集完成检索、生成与可观测链路量化分析；相比单一 dense 检索，混合检索 + RRF + rerank 使 **Recall@5 提升 X%**、**MRR@10 提升 X%**；L1/L2/L3 + Auto-merging 在跨段长文档问题上将 **回答准确率提升 X%**、**上下文完整率提升 X%**；动态触发 Step-Back/HyDE 使复杂问题 **Top-5 命中率提升 X%**；SSE 可观测推送将 **首 token 时间降低 X%**、实现 **100% tracing 覆盖**。

版本 B：偏工程能力

示例模板：

- 构建面向 RAG 系统的评测驱动优化体系，在检索、重排、长上下文恢复、查询重写与流式可观测链路等关键路径上完成多组消融实验；最终实现 **Recall@5 / MRR@10 / groundedness / TTFT** 等核心指标的系统性提升，并沉淀可复现的自动化评测流程。

---

13. 优先关注的 5 个核心数字

请优先确保以下 5 个数字可靠产出：

1. Recall@5
2. MRR@10
3. Cross-chunk answer accuracy
4. TTFT
5. Trace coverage rate

这 5 个数字优先用于简历第一版量化结果。

---

14. 附加要求

1. 不要伪造指标，不要编造数字
2. 若某项指标因数据或实现受限暂时无法计算，请在报告中明确说明原因
3. 所有中间结果尽量保存在本地，避免重复调用外部 API
4. 所有脚本应支持命令行参数和配置文件两种模式
5. 所有结果都要能追溯到具体实验配置

---

15. 本任务完成定义（Definition of Done）

当以下条件全部满足时，视为任务完成：

- 已成功下载并处理 LoTTE、RAGBench、custom 数据
- 已完成 retrieval / chunking / rewrite / rag / latency 五类实验
- 已输出样本级 jsonl 与汇总 csv
- 已生成 `results_summary.md`
- 已生成 `resume_bullets.md`
- 已在 README 中说明复现方法
- 已产出可直接写入简历的量化结果候选表述